BugSeer: Predicting Defect-Prone Software Modules Using Static Code Metrics

Ebrima Gajaga
Bachelor of Computer Applications
The Maharaja Sayajirao University of Baroda
Vadodara, India
Email: gajaga.dev@gmail.com
PRN: 8023058464

Pritibala Patel
Assistant Professor
Department of Computer Applications
The Maharaja Sayajirao University of Baroda
Email: preeti.patel-compapp@msubaroda.ac.in

Fig. 1. QR Code verifying student identity (MSU Baroda)

***Abstract*—Software defects remain a major challenge in large-scale systems, increasing maintenance cost and reducing reliability. This work investigates whether defect-prone code modules can be predicted using static code metrics derived directly from source code.**

We present BugSeer, a system that performs automated code analysis and predicts bug risk at the file level. A dataset is constructed from open-source Python repositories, where labels are approximated using heuristic rules based on structural complexity.

We extract features such as lines of code, cyclomatic complexity, nesting depth, and function-level characteristics. Logistic Regression and Random Forest models are evaluated.

Results show that Random Forest significantly outperforms Logistic Regression, achieving an F1-score of 0.86 and ROC-AUC of 0.90. Cross-validation confirms good generalization without severe overfitting.

These findings demonstrate that static code metrics alone provide meaningful signals for defect prediction.

***Index Terms*—Defect Prediction, Static Code Analysis, Machine Learning, Random Forest, Software Engineering**

I. Introduction

Modern software systems are increasingly complex, making defect detection and prevention challenging. Traditional approaches such as manual code review and testing are time-consuming and often insufficient.

Prior studies indicate that defect-prone modules exhibit measurable characteristics such as:

- High cyclomatic complexity
- Deep nesting structures
- Large code size

Research Question: Can defect-prone modules be predicted using only static code metrics?

II. Problem Formulation

Let:

$$F = \{f_1, f_2, ..., f_n\} \tag{1}$$

where F denotes the set of all source code files in the dataset, and f_i represents the ith file.

Each file is represented as a feature vector:

$$x_i \in \mathbb{R}^d \tag{2}$$

where x_i is a d-dimensional numerical representation of file f_i, and each dimension corresponds to a software metric.

We aim to learn a binary classification function:

$$g : \mathbb{R}^d \to \{0, 1\} \tag{3}$$

such that:

$$g(x_i) = \begin{cases} 1 & \text{if } f_i \text{ is predicted to be defect-prone} \\ 0 & \text{otherwise} \end{cases} \tag{4}$$

where:

- 1 indicates a high-risk (defect-prone) file
- 0 indicates a low-risk (non-defective) file

—

III. Dataset Construction

A. Repository Mining

Python repositories are collected using the GitHub API. Each repository is cloned and parsed locally to extract structural and complexity-based features.

—

B. Heuristic Labeling

Since ground-truth defect labels are not available, we construct proxy labels using a weighted risk function:

$$R(f_i) = \alpha_1 CC_i + \alpha_2 ND_i + \alpha_3 AFL_i + \alpha_4 LOC_i \tag{5}$$

where:

- CC_i = Cyclomatic Complexity of file f_i (measures number of independent execution paths)
- ND_i = Maximum Nesting Depth (deeply nested logic increases complexity)
- AFL_i = Average Function Length (average number of lines per function)
- LOC_i = Lines of Code (size of the file)
- $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ = learned or manually assigned weighting coefficients

The final binary label is assigned using a threshold τ:

$$y_i = \begin{cases} 1 & \text{if } R(f_i) \geq \tau \\ 0 & \text{otherwise} \end{cases} \tag{6}$$

where:

- $y_i = 1$ indicates a defect-prone file
- τ is a decision threshold controlling label sensitivity

Note: These labels are heuristic approximations and do not represent actual runtime bugs or confirmed defects.

—

C. Feature Extraction

Each file f_i is transformed into a feature vector using static code analysis.

Structural Features:

- $LOC(f_i)$: Total number of non-empty lines in file f_i
- $N_f(f_i)$: Number of functions defined in f_i
- $N_c(f_i)$: Number of classes defined in f_i

—

Complexity Features:

Cyclomatic complexity is defined as:

$$CC(f_i) = 1 + \sum \text{decision points in } f_i \tag{7}$$

where decision points include: `if`, `for`, `while`, `case`, and logical branches.

—

Nesting Depth:

- $ND(f_i)$ measures the maximum depth of nested control structures in f_i

—

Function Length:

$$AFL(f_i) = \frac{1}{N_f(f_i)} \sum_{j=1}^{N_f} L_j \tag{8}$$

where:

- L_j = number of lines in function j
- $N_f(f_i)$ = total number of functions in file f_i

—

IV. Model Design

A. Logistic Regression

$$P(y = 1|x) = \frac{1}{1 + e^{-w^T x}} \tag{9}$$

where:

- x = feature vector of a file
- w = learned weight vector
- $P(y = 1|x)$ = probability that file is defect-prone

—

B. Random Forest

$$\hat{y} = \text{mode}\{T_1(x), T_2(x), ..., T_k(x)\} \tag{10}$$

where:

- $T_k(x)$ = prediction from the kth decision tree
- k = total number of trees in the ensemble
- mode = majority voting across trees

Each tree is trained using:

- bootstrap sampling of training data
- random subset of features at each split

—

C. Model Selection

$$\theta^* = \arg\max_{\theta} F1(\theta) \tag{11}$$

where:

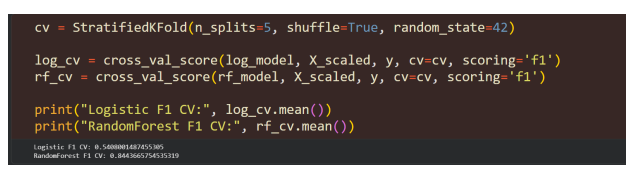
- θ = model hyperparameters
- $F1(\theta)$ = F1-score on validation set
- θ^* = optimal hyperparameter configuration

V. Evaluation Metrics

- Precision: $\frac{TP}{TP+FP}$
- Recall: $\frac{TP}{TP+FN}$
- F1-score
- ROC-AUC

VI. Results

A. Cross-Validation Comparison



```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

log_cv = cross_val_score(log_model, X_scaled, y, cv=cv, scoring='f1')
rf_cv = cross_val_score(rf_model, X_scaled, y, cv=cv, scoring='f1')

print("Logistic F1 CV:", log_cv.mean())
print("RandomForest F1 CV:", rf_cv.mean())
```

```
Logistic F1 CV: 0.5608001487455305
RandomForest F1 CV: 0.8443665754535319
```

Fig. 2. Cross-validation F1-score comparison between Logistic Regression and Random Forest

Cross-validation results show that Random Forest significantly outperforms Logistic Regression, achieving an F1-score of 0.84 compared to 0.54. This indicates better generalization and ability to capture non-linear patterns.

```
evaluate(log_model, "Logistic Regression")
evaluate(rf_model, "Random Forest")
```

```
Logistic Regression RESULTS
              precision    recall  f1-score   support

           0       0.81      0.91      0.86        46
           1       0.82      0.64      0.72        28

    accuracy                           0.81        74
   macro avg       0.81      0.78      0.79        74
weighted avg       0.81      0.81      0.81        74

Accuracy: 0.8108108108108109
Precision: 0.8181818181818182
Recall: 0.6428571428571429
F1: 0.72
ROC-AUC: 0.8773291925465838
```

Fig. 3. Logistic Regression performance metrics

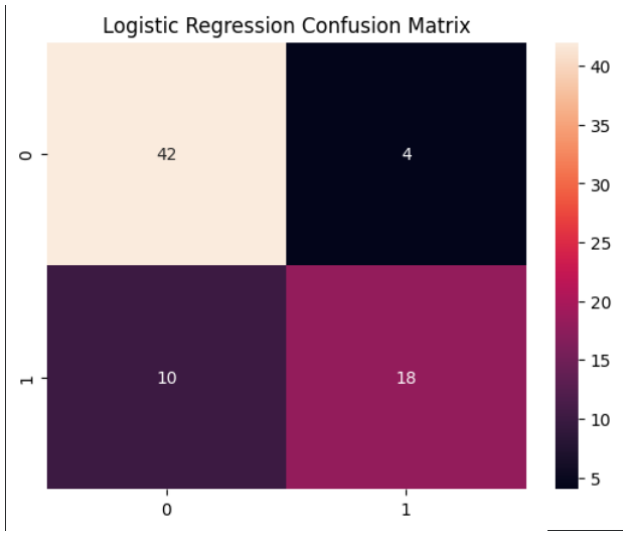


Fig. 4. Confusion Matrix for Logistic Regression

B. Logistic Regression Performance

Logistic Regression achieves moderate performance, with lower recall and F1-score, indicating underfitting due to its linear nature.

C. Random Forest Performance

Random Forest achieves strong predictive performance with:

- Accuracy: 0.89
- Precision: 0.83
- Recall: 0.89
- F1-score: 0.86
- ROC-AUC: 0.90

D. Feature Importance Analysis

Feature importance analysis reveals that:

- Cyclomatic complexity is the most influential feature
- Nesting depth and LOC also contribute significantly
- Function-level metrics provide additional predictive value

VII. Discussion

Results indicate that:

- Random Forest captures non-linear relationships better

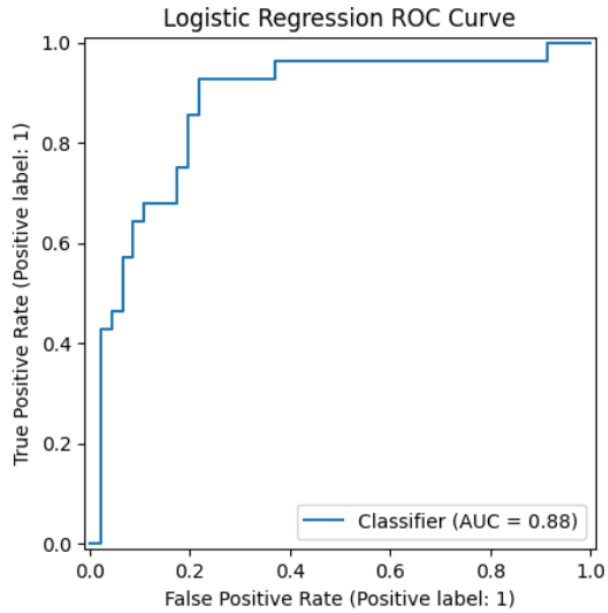


Fig. 5. ROC Curve for Logistic Regression

```
Random Forest RESULTS
              precision    recall  f1-score   support

           0       0.93      0.89      0.91        46
           1       0.83      0.89      0.86        28

    accuracy                           0.89        74
   macro avg       0.88      0.89      0.89        74
weighted avg       0.89      0.89      0.89        74

Accuracy: 0.8918918918918919
Precision: 0.8333333333333334
Recall: 0.8928571428571429
F1: 0.8620689655172413
ROC-AUC: 0.9041149068322981
```

Fig. 6. Random Forest performance metrics

- Complexity and nesting depth are dominant predictors
- Logistic Regression underfits the problem

VIII. System Architecture

The system consists of:

- Frontend: React dashboard
- Backend: Node.js API
- ML Service: FastAPI (Scikit-learn)

A. Project Availability

The implementation of BugSeer is publicly available:

- Source Code Repository: https://github.com/thanos14million605-Internship-2026/BugSeer

The system provides an end-to-end pipeline including feature extraction, model inference, and a web-based interface for real-time code risk analysis.

IX. Limitations

- Heuristic labels introduce noise
- No real defect ground truth
- Limited to Python codebases

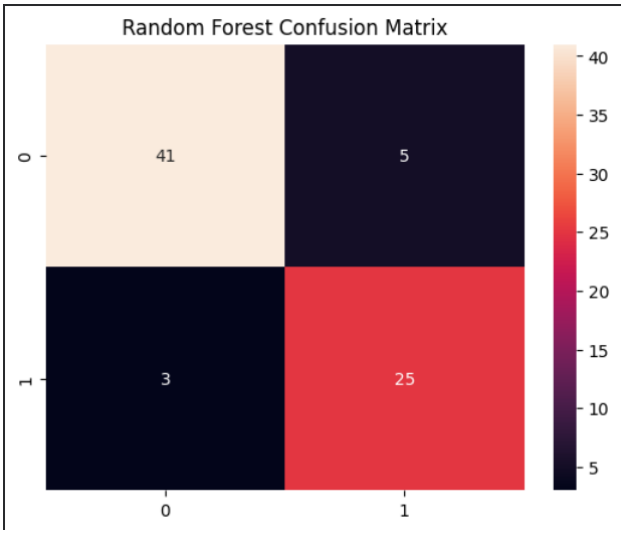


Fig. 7. Confusion Matrix for Random Forest

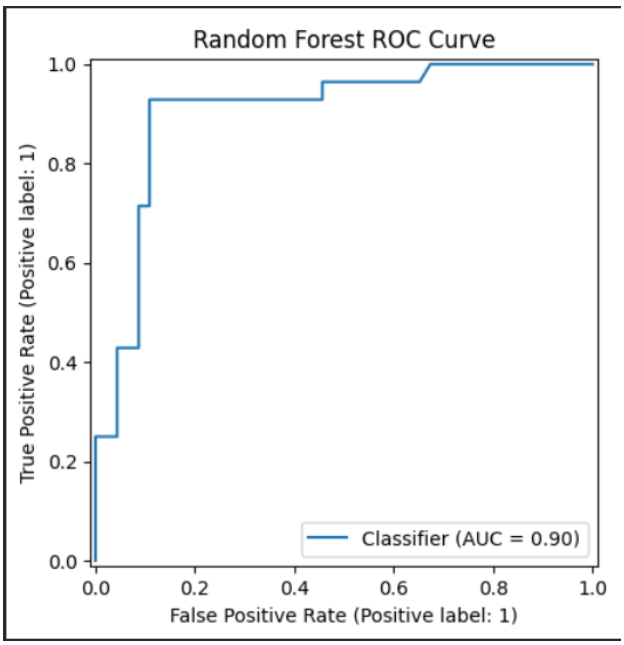


Fig. 8. ROC Curve for Random Forest

X. Conclusion

Static code metrics can effectively predict defect-prone modules. Random Forest achieves strong performance (F1 = 0.86), demonstrating the predictive power of structural code features.

XI. Future Work

- Incorporate version control features
- Use real defect datasets
- Explore deep learning on AST graphs

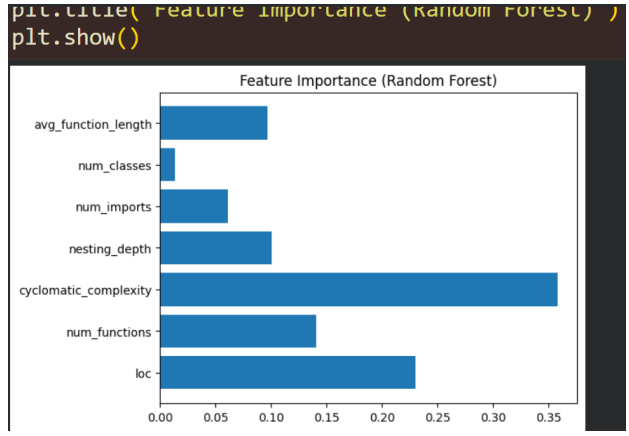


Fig. 9. Feature importance derived from Random Forest

Acknowledgment

This work was carried out as part of the Bachelor of Computer Applications program at The Maharaja Sayajirao University of Baroda (August 2023 – April 2026), under the guidance of Assistant Professor Pritibala Patel.

References

[1] T. J. McCabe, "A Complexity Measure," *IEEE Transactions on Software Engineering*, 1976.
[2] T. Hall, S. Beecham, and D. Bowes, "A Systematic Literature Review on Fault Prediction Performance," *IEEE Transactions on Software Engineering*, 2012.
[3] S. Lessmann et al., "Benchmarking Classification Models for Software Defect Prediction," *IEEE Transactions on Software Engineering*, 2008.
[4] T. Menzies et al., "Data Mining Static Code Attributes to Learn Defect Predictors," *IEEE Transactions on Software Engineering*, 2007.
[5] N. Nagappan and T. Ball, "Use of Relative Code Churn Measures to Predict System Defect Density," *ICSE*, 2005.
[6] L. Breiman, "Random Forests," *Machine Learning*, 2001.
[7] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, 2011.